

wpf总结

1. 基础

1. 引用基础类型

```
<!--引用常规类型命名空间-->
xmlns:system="clr-namespace:System;assembly=System.Runtime"

<!--定义资源-->
<Window.Resources>
    <models:Human x:Key="human" Children="Abc"></models:Human>
    <system:String x:Key="myString">Hello,world</system:String>
</Window.Resources>
<!--使用资源-->
<TextBlock x:Name="tb1" Text="{StaticResource myString}"></TextBlock>

<!--显示给属性赋值空值`{x:null}`-->
<!--TextWrapping 设置换行-->
```

2. 常用控件

ItemsControl 内容控件

```
<!--设置自动换行，起到使用wrapPanel控件作用-->
<ItemsControl.ItemsPanel>
    <!--ItemsPanelTemplate,用来更改ItemsControl中每项默认布局方式-->
    <ItemsPanelTemplate>
        <!--作为ItemsControl中项的宿主容器。添加到ListBox等ItemsControl中的所有项
        都将进入该面板进行布局-->
        <WrapPanel IsItemsHost="True" />
    </ItemsPanelTemplate>
</ItemsControl.ItemsPanel>
<!--更改ItemsControl中每个项的显示方式-->
<ItemsControl.ItemTemplate>
    <DataTemplate/>
</ItemsControl.ItemTemplate>
<!--用来设置每一个集合控件的Item的样式的属性-->
<ItemsControl.ItemContainerStyle>
    <Style TargetType="ListViewItem"/>
</ItemsControl.ItemContainerStyle>
```

DataGrid 列表控件

```
<DataGrid Grid.Row="1"
    <!--单元格Padding-->
    materialDesign:DataGridAssist.CellPadding="15,10,0,10"

    ItemsSource="{Binding LineItems}"
    <!--是否可以排序 -->
    CanUserSortColumns="False"
```

```

        <!--否可以在底部添加新行-->
        CanUserAddRows="False"
        <!--设置交替显示样式的数量,决定了有多少种不同的行样式会在DataGrid中循环使用-->
        AlternationCount="20"
        <!--行高-->
        RowHeight="38"
        <!--是否自动根据其绑定的数据源生成列-->
        AutoGenerateColumns="False"
        <!--分割线-->
        GridLinesVisibility="None"
        IsReadOnly="True">
    <DataGrid.Resources>
        <!--每列的标题信息-->
        <Style TargetType="{x:Type DataGridColumnHeader}" />
        <!--单元格样式-->
        <Style TargetType="DataGridCell"/>
        <!--行样式-->
        <Style TargetType="DataGridRow"/>
    </DataGrid.Resources>
</DataGrid>

```

3. Binding

- 对于 Binding 源本身就是数据且不需要Path来指明可以用 . 来代替属性值。
- RelativeSource 相对源, 当不明确作为Source对象的名字, 直到它与作为Binding目标的对象在 UI 布局上有对应关系时, 使用

```

<!--AncestorType 查找对象对应类型-->
<!--AncestorLevel 查找对象对应本控件辈分-->
Text="{ Binding RelativeSource={RelativeSource FindAncestor,AncestorType=
{x:Type Grid},AncestorLevel=1 },Path=Name}"
<!--当类型一致的时候还可以简写取消Path-->
Name="{ Binding RelativeSource={RelativeSource FindAncestor,AncestorType=
{x:Type Grid},AncestorLevel=1 }}"

<!--AncestorType 简化模式: 隐式查找最近的指定类型 (Grid) 祖先, 无需明确指定层级.-->
Text="{Binding Name, RelativeSource={RelativeSource AncestorType=Grid}}"

<!--自身Binding,也就是设置Height=width-->
Height="{ Binding RelativeSource={x:Static RelativeSource.Self },Path=width}"
<!--简写-->
Height="{ Binding RelativeSource=Self,Path=width}"

<!--绑定到该样式或模板宿主控件的属性-->
Background="{Binding Path=Background, RelativeSource={RelativeSource
TemplatedParent}}}"
<!--简写-->
Background="{Binding Path=Background, RelativeSource=TemplatedParent}"

<!--TemplateBinding,RelativeSource,都是用于ControlTemplate中-->
<!--TemplatedParent: 是完整的Binding-->

```

<!--TemplateBinding: 只能绑定到模板宿主控件的属性, 更简洁, 效率更高的单向绑定, 不支持转换器-->

<!--RelativeSource: 适合与复杂绑定-->

```
<Style TargetType="TextBox">
    <Setter Property="ToolTip">
        <Setter.Value>
            <!--ToolTip 属于一个单独的视觉树, 无法直接通过 RelativeSource 查找到宿主
            控件 (TextBox) 的 Text 属性-->
            <!--可以通过绑定的 PlacementTarget 来访问 ToolTip 的宿主控件-->
            <ToolTip>
                <ToolTip.Content>
                    <TextBlock Text="{Binding PlacementTarget.Text,
RelativeSource={RelativeSource AncestorType=ToolTip}}" />
                </ToolTip.Content>
            </ToolTip>
        </Setter.Value>
    </Setter>
</Style>
```

- BindingProxy: 绕过 WPF 的逻辑树层次限制, 从而使得其绑定源可以被全局访问

```
//表示这是一个可冻结的 WPF 对象
public class BindingProxy : Freezable
{
    // 依赖属性, 用于存储绑定的数据对象
    public static readonly DependencyProperty DataProperty =
        DependencyProperty.Register("Data", typeof(object),
typeof(BindingProxy), new PropertyMetadata(null));

    public object Data
    {
        get => GetValue(DataProperty);
        set => SetValue(DataProperty, value);
    }

    protected override Freezable CreateInstanceCore()
    {
        return new BindingProxy();
    }
}
```

```

<Window.Resources>
    <!-- 定义 BindingProxy,并绑定当前窗体的ViewMoodel -->
    <local:BindingProxy x:Key="Proxy" Data="{Binding}" />
</Window.Resources>

<DataTemplate>
    <StackPanel>
        <!-- 绑定当前项的属性 -->
        <TextBlock Text="{Binding Name}" />
        <!-- 使用 BindingProxy 绑定外部数据 -->
        <TextBlock Text="{Binding Data.GlobalMessage, Source={StaticResource
Proxy}}" />
    </StackPanel>
</DataTemplate>

```

```

<DataGrid.Resources>
    <Style TargetType="TextBlock">
        <Style.Triggers>
            <DataTrigger Binding="{Binding Text, RelativeSource=
{RelativeSource Self}, Converter={StaticResource lenStr},
ConverterParameter=1}" Value="true">
                <Setter Property="ToolTip">
                    <Setter.Value>
                        <ToolTip>
                            <ToolTip.Content>
                                <TextBlock TextWrapping="Wrap"
Width="300" Text="{Binding PlacementTarget.Text, RelativeSource=
{RelativeSource AncestorType=ToolTip}}" />
                            </ToolTip.Content>
                        </ToolTip>
                    </Setter.Value>
                </Setter>
            </DataTrigger>
        </Style.Triggers>
    </Style>
</DataGrid.Resources>

<!--在 DataTemplate 中显式引用 DataGrid.Resources 中定义的样式-->
<DataTemplate>
    <TextBlock Text="{Binding SampleCnName}"
        HorizontalAlignment="Center"
        VerticalAlignment="Center"
        Style="{StaticResource {x:Type TextBlock}}">
    </TextBlock>
</DataTemplate>

```

4. `TemplateBinding` 和 `TemplateParent` 通常与控件的模板（`ControlTemplate`）一起使用。可以使控件的内部视觉元素可以访问外部控件的属性。

- `TemplateBinding` 它默认绑定到模板的父控件。
- `TemplateParent` 通常用于模板内部的元素绑定到模板所属控件的属性。

5. 逻辑树，视觉树

- 逻辑树：控件的逻辑层次结构，沿逻辑树向上查找资源，反映代码中的内容关系（更简单）。
 - 视觉树：控件的实际呈现结构，路由事件通过视觉树传播。它包括控件本身以及所有用于渲染的子元素（更详细）。
6. 路由事件：是一种特殊的事件机制，可以沿着元素树传播，让你可以在父元素中集中处理来自子元素的事件，而无需为每个子元素单独编写代码。
- 冒泡事件 (Bubble)：从事件源向父控件传播。
 - 隧道事件 (Tunnel)：往往以 Preview 开头，从根元素向事件源传播，比冒泡事件先触发，可以用来过滤或拦截事件。
 - 直接事件 (Direct)：只在事件源控件上触发。

- + 如果同时处理隧道和冒泡事件，注意顺序以避免冲突。
- + 使用 `e.Handled = true;` 阻止事件进一步传播。
- + 在 WPF 自身的事件中，默认都是冒泡事件，只有以 Preview 开头的事件，才是隧道事件。

7. 附加属性

- **扩展控件功能**：附加属性可以为 WPF 中的控件或元素添加额外的功能，而不需要改变控件的代码。
- **设置控件间的关系**：附加属性可以用于设置控件之间的关系或提供某些布局和行为控制，如：
Canvas 控件上使用 `Canvas.Left`
- **提高灵活性和复用性**：使用附加属性，可以在多个地方复用同一个逻辑或行为。

8. 行为类

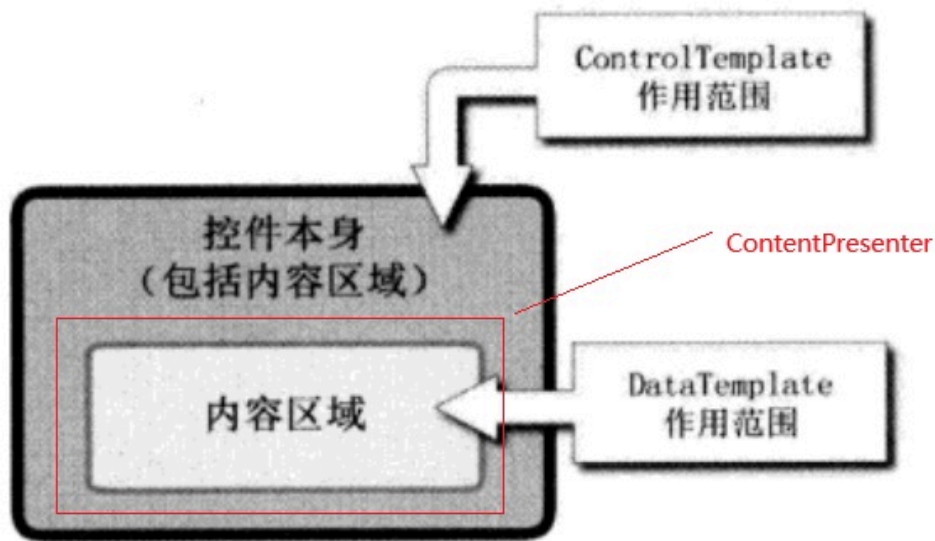
- 用于向 UI 元素添加交互逻辑而无需修改其代码。它提供了一种声明式的方式来附加行为到控件，从而简化了 XAML 的使用，并且提高了代码的可维护性和重用性。
- **提高代码复用性**：行为类的一个重要优点是其复用性。你可以定义一个行为，并在多个控件或元素上使用它，而不需要为每个控件编写单独的代码。

2. 模板

控件只是数据和行为的载体。至于它本身长成什么样子？数据长成什么样子都是靠 `Template` 生成的，决定控件外观的是 `ControlTemplate`，决定数据外观的是 `DataTemplate`，而他们两个都是由 `Template` 生成的控件。

只要能拿到 `Template` 找到其内部控件就不成问题

`ContentPresenter`：最常出现在模板（如 `ControlTemplate` 或 `DataTemplate`）中，用来作为内容显示的占位符。



2.1 ControlTemplate 控件模板—控件的外观

决定控件长成什么样子。

`ContentPresenter` 通常用于控件模板中，用来显示 `ContentControl` 的内容

```
<style>
  <setter property="Template" TargetType="{x:Type TextBox}">
    <setter.value>
      <ControlTemplate TargetType="{x:Type TextBox}">
        <!--Template内容 -->
        <ContentPresenter
          Content="{TemplateBinding Content}"
          ContentTemplate="{TemplateBinding ContentTemplate}" />
      </ControlTemplate>
    </setter.value>
  </setter>
</style>

<StackPanel>
  <UserControl x:Name="uc" Template="{StaticResource CTmp}" Margin="5">
</UserControl>
  <Button Content="Find By Name" Click="Button_Click"></Button>
</StackPanel>
```

寻找 `ControlTemplate` 中定义的子控件

```
private void Button_Click(object sender, RoutedEventArgs e)
{
  //根据Name获取子控件
  TextBox tb = this.uc.Template.FindName("textBox1", this.uc) as TextBox;
  tb.Text = "Hello WPF";
  //根据子控件找到父控件
  StackPanel sp = tb.Parent as StackPanel;
  //根据父控件找到其他子控件
  (sp.Children[1] as TextBox).Text = "Hello ControlTemplate";
  (sp.Children[2] as TextBox).Text = "I can find you!";
}
```

```

//查询控件模板
ControlTemplate template =
(ControlTemplate)this.FindResource("PageButtonTemplate3");
var btn1 = (Button)sp.FindName("btnFirst");
//重新赋值控件模板
btn1.Template = template;
}

```

2.2 DataTemplate 数据模板—数据的外观

控制一条数据展示成什么样子, DataTemplate 常用的地方有3处, 分别是

- 内容控件 ContentControl 的 ContentTemplate 属性, 相当于给 ContentControl 的内容穿衣服
- 列表控件 ItemsControl 的 ItemTemplate 属性, 相当于给 ItemsControl 的数据条目穿衣服。
- 表格控件 GridViewColumn 的 CellTemplate 属性, 相当于给 GridViewColumn 单元格里的数据穿衣服

DataTemplate 其载体一般是 ContentPresenter 对象上, ContentPresenter 只有 ContentTemplate 用来接收 DataTemplate, ContentPresenter 的用途就是, 承载 DataTemplate 生成的一组控件。

- ContentPresenter 可用于数据模板中以支持子模板, 例如在复杂布局中动态嵌套数据模板:

```

<Window.Resources>
    <!--DataTemplate-->
    <!--模板的DataContext就是Unit对象-->
    <DataTemplate DataType="{x:Type local:Unit}">
        <Grid>
            <StackPanel Orientation="Horizontal">
                <Grid>
                    <Rectangle Stroke="Yellow" Fill="Orange" width="{Binding
Price)"/>
                    <TextBlock Text="{Binding Year}"/>
                </Grid>
            </StackPanel>
        </Grid>
    </DataTemplate>
    <!--数据源-->
    <c:ArrayList x:Key="ds">
        <local:Unit Year="2001年"Price="100"/>
        <local:Unit Year="2002年"Price="120"/>
        <local:Unit Year="2003年"Price="140"/>
        <local:Unit Year="2004 年"Price="160"/>
    </c:ArrayList>

```

<!--在 WPF 中, ContentPresenter 的作用是: 把某个控件的 Content 显示出来。-->
 <!--如果宿主控件没有Content属性, 可以用它绑定其他数据-->
 <ContentPresenter Content="{Binding DynamicContent}" />

```

</Window.Resources>
<StackPanel>
    <ListBox ItemsSource="{StaticResource ds}" />
    <ComboBox ItemsSource="{StaticResource ds}" Margin="5" />
</StackPanel>

<StackPanel>
    <ContentPresenter x:Name="cp" ContentTemplate="{StaticResource CTmp}"
Margin="5"></UserControl>
    <Button Content="Find By Name" Click="Button_Click"></Button>
</StackPanel>

```

寻找 `DataTemplate` 中定义的子控件

```

private void Button_Click(object sender, RoutedEventArgs e)
{
    TextBlock tb = this.cp.ContentTemplate.FindName("textBlockName", this.cp) as
TextBlock;

    Student stu=this.cp.Content as Student;
    MessageBox.Show(stu.Name);
}

```

3. 触发器

```

<Style>
    <Style.Triggers>
        <!--单条件触发器-->
        <Trigger Property="IsCheck" value="true">
            <Trigger.Setters>
                <Setter Property="FontSize" Value="20"></Setter>
                <!--.....-->
            </Trigger.Setter>
        </Trigger>
        <!--多条件触发器-->
        <MultiTrigger>
            <MultiTrigger.Conditions>
                <Condition Property="IsCheck" value="true"></Condition>
                <Condition Property="Content" value="测试"></Condition>
            </MultiTrigger.Conditions>
            <MultiTrigger.Setters>
                <Setter Property="FontSize" value="20"></Setter>
                <!--.....-->
            </MultiTrigger.Setters>
        </MultiTrigger>

        <!--单条件数据触发器-->
        <DataTrigger Binding="{TemplateBinding Name}" value="测试">
            <Setter Property="FontSize" Value="20"></Setter>
            <!--设置事件-->
            <EventSetter Event="Loaded" Handler="ComboBox_Loaded"/>

```



```

</DataTrigger>

<!--多条件数据触发器-->
<MultiDataTrigger>
    <MultiDataTrigger.Conditions>
        <Condition Binding="{TemplateBinding Id}" value="1"></Condition>
        <Condition Binding="{TemplateBinding Name}" value="测试">
</Condition>
    </MultiDataTrigger.Conditions>
    <MultiDataTrigger.Setters>
        <Setter Property="FontSize" value="20"></Setter>
    </MultiDataTrigger.Setters>
</MultiDataTrigger>
</Style.Trigger>
</Style>

```

4. 占位符

```

<ControlTemplate x:Key="bottomValidTmp">
    <DockPanel>
        <TextBlock DockPanel.Dock="Bottom" Foreground="Red"
            Text="{Binding ElementName=adornedElement,
Path=AdornedElement.(Validation.Errors)[0].ErrorContent}"></TextBlock>
        <Border BorderBrush="Red" BorderThickness="0">
            <!--装饰元素占位符,确保被装饰的元素在其正确的位置上显示,并且可以被其他Adorner元素环绕或覆盖。-->
            <!--Adorner可以用于在现有控件上添加额外的视觉效果或功能,而不会改变控件的布局或行为-->
            <AdornedElementPlaceholder Name="adornedElement">
</AdornedElementPlaceholder>
        </Border>
    </DockPanel>
</ControlTemplate>

```